

Metodología SaaS Factory

Antigravity

Mayo 2026

Índice

Tabla de Contenidos	3
1. Metodología SaaS Factory	4
Qué es	4
Por Qué Funciona	4
2. Fase 1: DISCOVERY (Semana 1-2)	5
Qué Pasa	5
Entrevista Profunda	5
Documentación: BUSINESS_LOGIC.md	5
Visión	5
Usuarios	5
Features MVP	5
Out of Scope	5
Requisitos Técnicos	5
Entidades Clave	5
Flujos Principales	5
Métricas de Éxito	5
Deliverable	5
3. Fase 2: ARCHITECTURE (Semana 2-3)	6
Qué Pasa	6
Análisis Arquitectónico	6
Estructura de Carpetas	6
Entidades Base	6
Deliverable	6
4. Fase 3: SPRINT 1 - AUTH & ONBOARDING (Semana 3-4)	7
Qué Pasa	7
Setup Inicial	7
Auth Completa	7
Testing	7
Deliverable	7
5. Fase 4: SPRINT 2-N - FEATURES (Semana 5+)	8
Qué Pasa	8
Ciclo Feature Típica	8
Ejemplo: Feature “Crear Pedido”	8
Testing Continuo	8
Deliverable	8
6. Fase 5: LAUNCH & OPERATIONS (Semana N+1)	9
Qué Pasa	9

Pre-Launch Checklist	9
Production Setup	9
Operaciones Continuas	9
Semanal	9
Mensual	9
Trimestral	9
Anual	9
Deliverable	9
7. Principios Clave	10
1. User-Centric	10
2. Iterativo	10
3. Automatizado	10
4. Documentado	10
5. Escalable	10
6. Seguro por Defecto	10
7. Colaborativo	10
8. Comparación: Antes vs Después	11
Métrica de Éxito	11
Conclusión	11

Tabla de Contenidos

1. **Introducción** — Metodología probada
2. **Fase 1: Discovery** — Entender el problema
3. **Fase 2: Architecture** — Diseñar la solución
4. **Fase 3: Sprint Auth** — Autenticación completa
5. **Fase 4: Sprints Features** — Construcción iterativa
6. **Fase 5: Launch** — Lanzamiento a producción
7. **Principios Clave** — Lo que hace diferente
8. **Comparación** — Antes vs después

1. Metodología SaaS Factory

Qué es

SaaS Factory es una metodología **ágil y centrada en el usuario** que garantiza:

- Claridad en requisitos desde el inicio
- Entrega de valor en cada fase
- Código limpio y mantenible
- Testing automático en cada paso
- Comunicación continua con stakeholders

Por Qué Funciona

No es “agile genérico”. Es la ejecución pura de valor con herramientas modernas específicamente elegidas.

2. Fase 1: DISCOVERY (Semana 1-2)

Qué Pasa

El usuario cuenta **QUE** quiere. Nosotros descubrimos **COMO** construirlo.

Entrevista Profunda

Respondemos preguntas críticas:

- ¿Cuál es el problema que resolvemos?
- ¿Quiénes son los usuarios reales?
- ¿Cuál es el MVP (producto mínimo viable)?
- ¿Qué NO incluye el MVP? (Crucial para focus)
- ¿Cuáles son los requisitos técnicos?
- ¿Presupuesto y timeline?

Documentación: BUSINESS_LOGIC.md

Generamos documento que describe:

Visión

El QUE y POR QUE del producto.

Usuarios

Quiénes lo usan y sus necesidades reales.

Features MVP

Las features que SÍ incluyen (máximo 5 para focus).

Out of Scope

Lo que NO incluimos (importante para no creep).

Requisitos Técnicos

Escalabilidad, seguridad, performance esperada.

Entidades Clave

Estructuras de datos principales.

Flujos Principales

Cómo los usuarios interactúan con el sistema.

Métricas de Éxito

Cómo medimos que funcionó.

Deliverable

BUSINESS_LOGIC.md + Design System elegido

3. Fase 2: ARCHITECTURE (Semana 2-3)

Qué Pasa

Diseñamos la arquitectura técnica basada en requisitos reales.

Análisis Arquitectónico

- Estructura de carpetas (feature-first)
- Componentes principales
- Entidades de BD y relaciones
- Estrategia de seguridad (RLS, auth)
- Optimizaciones de performance
- Flujos de datos

Estructura de Carpetas

```
src/  
  app/  
    (auth)/      # Rutas auth  
    (main)/      # Rutas principales  
    layout.tsx  
  features/      # Organizadas por funcionalidad  
    [feature]/  
      components/  
      hooks/  
      services/  
      types/  
      store/  
  shared/        # Código reutilizable  
    components/  
    hooks/  
    lib/  
    types/
```

Entidades Base

- Tabla **users** — Usuarios del sistema
- Tabla **profiles** — Información extendida
- Tablas de feature específicas
- Todas con RLS habilitado

Deliverable

ARCHITECTURE.md + ERD (Entity-Relationship Diagram)

4. Fase 3: SPRINT 1 - AUTH & ONBOARDING (Semana 3-4)

Qué Pasa

Implementamos autenticación y onboarding completo.

Setup Inicial

1. Crear proyecto

```
npx create-next-app@latest [nombre] --typescript
```

2. Instalar SaaS Factory

```
npm install saas-factory
```

3. Inicializar estructura

```
npx saas-factory init
```

Auth Completa

- Email + Password
- OAuth (Google, GitHub)
- User Profiles
- RLS en tablas
- Session Management
- Email verification

Testing

```
npm run test           # Tests unitarios
```

```
npm run test:e2e       # Tests E2E
```

```
npm run test:coverage  # Coverage report
```

Deliverable

App con auth funcionando, tests pasando

5. Fase 4: SPRINT 2-N - FEATURES (Semana 5+)

Qué Pasa

Construimos las features principales del MVP, una a una.

Ciclo Feature Típica

Para cada feature:

1. **Crear tipos** — `types/feature.ts`
2. **Crear DB schema** — Migraciones SQL
3. **Crear API** — Server Actions
4. **Crear componentes** — UI con React
5. **Crear tests** — Cobertura 80%+
6. **Deploy** — Automático a Vercel

Ejemplo: Feature “Crear Pedido”

```
features/  
  orders/  
    components/  
      CreateOrderForm.tsx  
      OrderList.tsx  
      OrderDetail.tsx  
    hooks/  
      useCreateOrder.ts  
      useOrders.ts  
    services/  
      orderService.ts  
    types/  
      order.ts  
    store/  
      orderStore.ts  
    tests/  
      OrderForm.test.tsx  
      orderService.test.ts
```

Testing Continuo

Cada PR:

- Tests unitarios
- Tests E2E
- Linting
- Type checking
- Build validation

Deliverable

Feature completa, testeada, deployada

6. Fase 5: LAUNCH & OPERATIONS (Semana N+1)

Qué Pasa

Preparamos la salida a producción y establecemos operaciones.

Pre-Launch Checklist

- Todos los tests pasando
- Performance optimizado (Lighthouse 90+)
- Seguridad validada (RLS, CORS, secrets)
- Backups configurados
- Alertas configuradas
- Documentación completada
- Usuarios creados en producción

Production Setup

```
# Environment variables en Vercel
NEXT_PUBLIC_API_URL=https://app.example.com
DATABASE_URL=postgresql://...
JWT_SECRET=***
```

```
# Dominio custom
Vercel DNS -> example.com
```

```
# Monitoring
Vercel Analytics + Error tracking
```

Operaciones Continuas

Semanal

Revisar logs y alertas

Mensual

Actualizar dependencias

Trimestral

Auditoría de seguridad

Anual

Actualizar dependencias mayores

Deliverable

App en producción, estable, monitoreado

7. Principios Clave

1. User-Centric

Todo empieza con el usuario. Sus necesidades guían el desarrollo.

2. Iterativo

Entrega de features en ciclos cortos (2 semanas máximo).

3. Automatizado

Tests, builds y deployments todos automáticos.

4. Documentado

El código se documenta a sí mismo (TypeScript).

5. Escalable

La arquitectura crece con el producto.

6. Seguro por Defecto

Seguridad integrada desde el inicio.

7. Colaborativo

Comunicación constante entre todos.

8. Comparación: Antes vs Después

Métrica de Éxito

Aspecto	Sin SaaS Factory	Con SaaS Factory
MVP Time	8-12 semanas	2-3 semanas
Testing	Manual	Automático
Deploy	Servidor manual	30 segundos
Escalar	Reescribir código	Cero cambios
Seguridad	Agregar después	Integrada
Ops Time	20 % del tiempo	0 % (automático)

Conclusión

La **Metodología SaaS Factory** es un blueprint comprobado para construir productos de calidad rápidamente.

Cuando usas SaaS Factory:

- Entregas faster
- Entregas better
- Entregas sin stress

Antigravity | Consultoría de Sistemas & Transformación Digital | Mayo 2026

Donde la excelencia técnica se encuentra con el impacto empresarial